DS-GA 1003 Machine Learning: Homework 2

Due 11.59 p.m. EST, March 26, 2024 on Gradescope

(Jiasheng Ni)
(collaborators if any)

We encourage LaTeX-typeset submissions but will accept quality scans of hand-written pages.

1 Very Random Forest

Consider building a random forest by both subsampling the data and choosing a single feature per tree randomly. For example, consider a dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), ..., (\mathbf{x}_N, y_N)\}$ where $\mathbf{x}_i \in \mathbb{R}^D$ and $y_i \in \mathbb{R}$ for $i = 1, .., N$. We will carry out the following procedure:

1. Randomly sample one feature index $j \in \{1, \ldots, D\}$.
2. Draw a sample of the data $\mathcal{D}_{\mathbf{k}}$ of size $M \leq N$ with replacement. These datapoints will have indices $\mathbf{k} = k_1, \ldots, k_M$.
3. Keep only the jth feature of the M samples: i.e. letting $x_{i,j}$ be the ith datapoint and j^{th} feature, we use data
$$\mathcal{D}^j_{\mathbf{k}} = \{(x_{(k_1,j)}, y_{(k_1)}), ..., (x_{(k_M,j)}, y_{(k_M)})\}$$
4. Build a decision tree on $\mathcal{D}^j_{\mathbf{k}}$.
5. Repeat the above process R times so that r^{th} tree T_r uses feature $j^{(r)}$ and data $\mathbf{k}^{(r)}$ for $r \in \{1, \ldots, R\}$. Using this notation, the prediction of the rth tree on new input $\mathbf{x}^\star$ is $T_r(\mathbf{x}^\star; \mathcal{D}^{j^{(r)}}_{\mathbf{k}^{(r)}})$.
6. Average these random trees to construct the random forest. That is, for input $\mathbf{x}^\star$, the random forest predicts $\hat{y} = \frac{1}{R}\sum_{r=1}^{R} T_r(\mathbf{x}^\star; \mathcal{D}^{j^{(r)}}_{\mathbf{k}^{(r)}})$.

Let us call this model a Very Random Forest (VRF). In this question, we will characterize the *bias* of an VRF (in the context of bias-variance tradeoff).

(A) Write down the two term that have to be equal for a model to be unbiased. One term should be some statistic of the true data distribution and the other should be a statistic of the model output.

Solution. The two terms that have to be equal are

$$\mathbb{E}_{\mathbf{K},\mathbf{J}}\left[T_r\left(\mathbf{x}^\star; \mathcal{D}^J_{\mathbf{K}}\right)\right] = \mathbb{E}_{\mathbf{Y}|\mathbf{X}}\left[Y \mid \mathbf{X} = \mathbf{x}^\star\right]$$

The left expression is the prediction output of our VRF model while the right one is the true value for the regression task. Once these two quantities are equal, the model is unbiased. □

(B) Let us first consider a single decision tree $T(\mathbf{x}, \mathcal{D}^j)$ that uses only a single feature j but the **entire** dataset (i.e. all N data points). Assuming that the tree was trained by minimizing squared loss, what will be the tree's prediction for a test point $\mathbf{x}^\star$? I.e. write down the function that $T(\mathbf{x}^\star, \mathcal{D}^j)$ corresponds to.

Solution. Suppose the set of indices of the training points in the leaf node that $\mathbf{x}^\star$ fall into, using the j-th feature as $C(\mathbf{x}^\star, j) = \{i \mid x_{i,j} \text{ is in the same leaf node as } x^\star\}$.
Then the prediction of the tree for $\mathbf{x}^\star$ will be defined as:

$$T\left(\mathbf{x}^\star, \mathcal{D}^j\right) = \frac{1}{|C(\mathbf{x}^\star, j)|} \sum_{i \in C(\mathbf{x}^\star, j)} y_i \tag{1}$$

$$= \frac{\sum_{i=1}^{N} \mathbf{1}\{i \in C(\mathbf{x}^\star, j)\} y_i}{|C(\mathbf{x}^\star, j)|} \tag{2}$$

where $|C(\mathbf{x}^\star, j)|$ denotes the cardinality of the set that $\mathbf{x}^*$ falls in using the j-th feature. □

(C) Now, express the model-dependent term that you wrote in part (A) in terms of your answer in (B).

Solution. The model-dependent term becomes:

$$\begin{aligned}\mathbb{E}_{\mathbf{K},\mathbf{J}}\left[T_r\left(\mathbf{x}^\star; \mathcal{D}_{\mathbf{k}}^J\right)\right] &= \mathbb{E}_J\left[\mathbb{E}_{\mathbf{K}|\mathbf{J}}\left[T\left(\mathbf{x}^\star, \mathcal{D}_{\mathbf{K}}^J\right)\right]\right] \\ &= \mathbb{E}_J\left[\mathbb{E}_{\mathbf{K}}\left[T\left(\mathbf{x}^\star, \mathcal{D}_{\mathbf{K}}^J\right)\right]\right]\end{aligned}$$

where K and J should be independent. □

(D) As $N \to \infty$, what data-dependent function does your answer in part (C) converge to? You may assume that your VRF has sufficient capacity to model this function arbitrarily well. Describe in words the source of bias.

Solution. The data-dependent function in the case in (B) should converges to the conditional expectation $\mathbb{E}[Y|X_j = \mathbf{x}_j^\star]$, which is the average target value within the partition that contains $\mathbf{x}^\star$ using the j-th feature.
When we are using the whole VRF, each tree is built on a different feature randomly, in this case, the function converges to the $\mathbb{E}_J[\mathbb{E}[Y|X_J = \mathbf{x}_J^\star]]$, which is the average of the average prediction across different random features. □

(E) Let us build another VRF. Each tree in this forest is built by randomly sampling *two different features* instead of one in step 1. As $N \to \infty$, what data-dependent function does your answer in part (C) converge to? How does using two different features instead of one reduce the bias of VRF?

Solution. If we are using two features, with the model in (B), then the data-dependent function should converge to $\mathbb{E}[Y|X_m = \mathbf{x}_m^\star, X_n = \mathbf{x}_n^\star]$. If we are using the whole VRF that randomly selects two features for each tree, the data-dependent function should converge to $\mathbb{E}_{M,N}[\mathbb{E}[Y|X_M = \mathbf{x}_M^\star, X_N = \mathbf{x}_N^\star]]$, which is the average of the conditional expectations over all possible pairs of features. □

(F) Compare the bias and variance of the VRF's with the traditional random forest, where we select a random subset of the data and a random subset of features to build each tree.

Hint: Look at the generalization bound from the lecture on random forests. You only need to look at the final result, not the derivation.

Solution. (a) Bias: Using two-feature VRF can make the loss decrease faster at each choice of the feature and split point. Moreover, two-feature model allows the model to consider the interaction of the features instead of independence among features.

(b) Variance: The variance in the VRF model will be higher since we use the entire data when we fit a single decision tree. Meanwhile, the traditional random forest, due to subsampling, will have lower variance in general.

□

2 Conditional Modelling with Gaussians

For any finite set of points $\mathcal{D} = \{(\mathbf{x}_1, y_1), ..., (\mathbf{x}_N, y_N)\}$, assume the following conditional model:

$$\begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \Bigg| \mathbf{x}_1, ..., \mathbf{x}_N \sim \mathcal{N}(\vec{0}, K)$$

Here, K is a covariance matrix such that $K_{ij} := k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\frac{1}{2}\|\mathbf{x}_i - \mathbf{x}_j\|^2)$. You may assume that K is positive-definite for any dataset $\mathcal{D}$.

(A) For any set of $(N+1)$ points $\mathbf{x}_1, \ldots, \mathbf{x}_{N+1}$, write the down the distribution of $y_1, \cdots y_N, y_{N+1} \mid \mathbf{x}_1, \cdots \mathbf{x}_N, \mathbf{x}_{N+1}$ under the assumed model.

Solution. This is a noiseless gaussian process regression model where the distribution $y_1, \cdots y_N, y_{N+1} \mid \mathbf{x}_1, \cdots \mathbf{x}_N, \mathbf{x}_{N+1}$ is assumed also to be multivariate gaussian where $K_{N+1} = \begin{bmatrix} K & K(\vec{x}, \mathbf{x}_{N+1}) \\ K(\mathbf{x}_{N+1}, \vec{\mathbf{x}}) & K(\mathbf{x}_{N+1}, \mathbf{x}_{N+1}) \end{bmatrix}$ and

$$K \in \mathbf{R}^{N\times N} \text{ such that } K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$$
$$K(\vec{\mathbf{x}}, \mathbf{x}_{N+1}) \in \mathbf{R}^{N\times 1} \text{ such that } (K(\vec{\mathbf{x}}, \mathbf{x}_{N+1}))_{ij} = k(\mathbf{x}_i, \mathbf{x}_{N+1,j})$$
$$K(\mathbf{x}_{N+1}, \vec{\mathbf{x}}) \in \mathbf{R}^{1\times N} \text{ such that } (K(\mathbf{x}_{N+1}, \vec{\mathbf{x}}))_{ij} = k(\mathbf{x}_{N+1,i}, \mathbf{x}_j)$$
$$K(\mathbf{x}_{N+1}, \mathbf{x}_{N+1}) \in \mathbf{R}^{1\times 1} \text{ such that } (K(\mathbf{x}_{N+1}, \mathbf{x}_{N+1}))_{ij} = k(\mathbf{x}_{N+1,i}, \mathbf{x}_{N+1,j}).$$
$$\vec{\mathbf{x}} = \left[\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\right]$$

and that

$$\begin{bmatrix} y_1 \\ \vdots \\ y_{N+1} \end{bmatrix} \Bigg| \mathbf{x}_1, \ldots, \mathbf{x}_{N+1} \sim \mathcal{N}(\overrightarrow{0}, K_{N+1})$$

□

(B) Given a dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), ..., (\mathbf{x}_N, y_N)\}$ and a test point $\mathbf{x}_{N+1}$, how do you make a prediction using this model?

Solution. The prediction of y_{N+1} will be the mean following MVG posterior distribution:

$$y_{N+1}|\mathbf{x}_{N+1}, \vec{\mathbf{x}}, \vec{\mathbf{y}} \sim \mathcal{N}(K(\mathbf{x}_{N+1}, \vec{\mathbf{x}})^\top K^{-1}\vec{\mathbf{y}}, K_{\mathbf{x}_{N+1},\mathbf{x}_{N+1}} - K(\mathbf{x}_{N+1}, \vec{\mathbf{x}})^\top K^{-1} K(\mathbf{x}_{N+1}, \vec{\mathbf{x}}))$$

by bayes rule for MVG, where $\vec{\mathbf{y}} = \left[\mathbf{y}_1, \mathbf{y}_2, \cdots, \mathbf{y}_N\right]^\top$ such that $\widehat{\mathbf{y}_{N+1}} = K(\mathbf{x}_{N+1}, \vec{\mathbf{x}})^\top K^{-1}\vec{\mathbf{y}}$. In short, to predict $\mathbf{y}_{N+1}$, we use the mean of the marginal distribution of y_{N+1} as the point estimate and the variance to compute the confidence interval for this point estimate. This means for different test point x^*, we will have different width of confidence interval, which is different from linear regression where the width of the confidence interval is the same across all test points. □

(C) Suppose that the test point $\mathbf{x}_{N+1}$ is an outlier, e.g. assume $\min_{1\le i\le N}\|\mathbf{x}_{N+1} - \mathbf{x}_i\| > 1000$. Using your answer in part (B), what is your prediction for $\mathbf{x}_{N+1}$?

Solution. Given that $\min_{1\le i\le N}\|\mathbf{x}_{N+1} - \mathbf{x}_i\| > 1000$, we find that $K(\mathbf{x}_{N+1}, \vec{\mathbf{x}})_{1,j} = \exp(-\frac{1}{2}\|\mathbf{x}_{N+1} - \mathbf{x}_j\|^2)$ will be close to a zero matrix, which will make our point estimate $\widehat{\mathbf{y}_{N+1}}$ arbitrarily close to 0. Meanwhile,the variance of the $\widehat{\mathbf{y}_{N+1}}$ will be close to $K(\mathbf{x}_{N+1}, \mathbf{x}_{N+1}) = \exp(-\frac{1}{2}\|\mathbf{x}_{N+1} - \mathbf{x}_{N+1}\|^2) = 1$ since $K(\mathbf{x}_{N+1}, \vec{\mathbf{x}})^\top K^{-1} K(\mathbf{x}_{N+1}, \vec{\mathbf{x}})$ will be close to 0. In short, the model will predict the y_{N+1} to be normally distributed with mean arbitrarily close to 0 and variance arbitrarily close to 1. □

(D) Now, imagine you fit a flexible neural network f_θ on the same dataset $\mathcal{D}$. Consider the same outlier $\mathbf{x}_{N+1}$ as in part (C). What can you say about the prediction $f_\theta(\mathbf{x}_{N+1})$? Does your answer change depending on your choice of network f_θ?

Hint: Think about the constraints a neural network may impose on the prediction for $\mathbf{x}_{N+1}$.

Solution. Since neural network models tend to have huge amount of parameters, it is very easy to overfit the training data and thus neural networks are not good at extrapolating the test points that have different distribution as the training dataset. Since here $\mathbf{x}_{N+1}$ is an outlier, the prediction $f_\theta(\mathbf{x}_{N+1})$ would be biased towards the training dataset and have high variance at the test point, making the prediction unreliable. □

(E) Compare the predictions you made in parts (C) and (D). Are they the same? If not, explain the difference. If one has a problem, suggest a way to solve it.

Solution. (a) Gaussian process regression model is a non-parametric, MVG-based model which come equipped with the uncertainty in their prediction. In other words, the GP model gives the entire posterior distribution $P_{Y|X}$ and use $E[Y|X]$ as the point estimate while the neural network is a parametric model that aims to directly predict $E[Y|X]$, without any knowledge of the posterior distribution.

(b) GP model quantifies the stability and reliability of prediction of outliers by the variance of posterior distribution, while neural network quantifies it using loss function.

(c) Since the neural network doesn't include any estimate on the uncertainty of output, we could integrate some of the bayesian ideas into it. For example, we could incorporate Gaussian process layers, which directly model the uncertainty in a similar way to GPR. This approach combines the flexibility of neural networks with the probabilistic nature of Gaussian processes. Moreover, like in regularized linear regression, instead of viewing the parameters as fixed quantities, we could add a prior distribution to the weights and bias of neural network(BNN's idea) to better model the distribution of the output.

□

3 Neural Networks for Reconstruction

In class, we saw how neural networks are a flexible model class that can be used for supervised regression or classification tasks, i.e. predicting y from $\mathbf{x}$. Let us consider a neural network that tries to predict $\mathbf{x}$ from $\mathbf{x}$ instead. In other words, given an input $\mathbf{x} \in \mathbb{R}^D$, the neural network will be trained to reconstruct the same $\mathbf{x}$.

Let us consider the network's architecture as follows:

$$\mathbf{h} = a(\mathbf{V}\mathbf{x})$$
$$\hat{\mathbf{x}} = \mathbf{W}\mathbf{h}$$

Here, our network contains a single hidden layer of size H, hence $\mathbf{V}$ is a $H \times D$ matrix and $\mathbf{W}$ is a $D \times H$ matrix. $a(\cdot)$ is an element-wise non-linear activation function that we will leave undecided for now. We have omitted bias terms to simplify the math.

I Gradients

(A) Write down the loss function L of our network for a single data point $\mathbf{x}$, assuming element-wise squared loss. Compute the derivative $\frac{\mathrm{d}L}{\mathrm{d}\mathbf{h}}$. Do not leave any symbolic expressions of the form $\frac{dy}{dx}$, i.e. actually compute the derivative in terms of known quantities like $\mathbf{W}$ or $\mathbf{x}$.

Solution.

$$\begin{aligned} L &= \sum_{i=1}^{D} (\hat{x}_i - x_i)^2 \\ &= \sum_{i=1}^{D} ((Wh)_i - x_i)^2 \\ &= \sum_{i=1}^{D} \left(w_i^\top h - x_i\right)^2 \end{aligned}$$

where ω_i is i-th row of W

$$\begin{aligned} \frac{dL}{dh} &= \sum_{i=1}^{D} 2\left(\omega_i^\top h - x_i\right) \cdot \omega_i \\ &= \sum_{i=1}^{D} 2\omega_i\omega_i^\top h - \sum_{i=1}^{D} x_i \cdot \omega_i \\ &= 2W^\top W h - W^\top x \end{aligned}$$

□

(B) Let $v := \mathbf{V}_{1,1}$ be the element in the first row and first column of $\mathbf{V}$. Compute the derivative $\frac{\mathrm{d}\mathbf{h}}{\mathrm{d}v}$. You can assume that $\frac{\mathrm{d}a(x)}{\mathrm{d}x} = a'(x)$. Next, using the chain rule and your answer in (A), compute the derivative $\frac{\mathrm{d}L}{\mathrm{d}v}$. As in (A), do not leave any symbolic expressions of the form $\frac{dy}{dx}$.

Solution.

$$\left(\frac{dh}{dv}\right)_i = \frac{da}{du_i} \cdot \frac{du_i}{dv}$$

where $u_i = v_i^\top x$ and v_i is i-th row of V

$$\therefore \frac{du_i}{dv} = \begin{cases} x_1, & i = 1 \\ 0, & i \neq 1 \end{cases}$$

$$\frac{da}{du_i} = a'(u_i)$$

$$\therefore \left(\frac{dh}{dv}\right)_i = \begin{cases} a'((Vx)_1) \cdot x_1, i = 1 \\ 0, \quad i \neq 1 \end{cases}$$

$$\therefore \left(\frac{dL}{dv}\right)_i = \frac{dl}{dh} \cdot \left(\frac{dh}{dv}\right)_i$$

$$= 2W^\top W(h - x) \cdot \begin{cases} a'((Vx)_1) \cdot x_1 & , i = 1 \\ 0 & , i \neq 1 \end{cases}$$

where $(Vx)_1 = \sum_{j=1}^{D} V_{1,j} x_j$. □

II Representation

(C) How does our choice of H affect the neural network's performance? Assuming that our optimization procedure successfully finds a network with minimum loss, describe the characteristics of an optimal network in the cases: (i) $H > D$, (ii) $H = D$, and (iii) $H < D$.

In each of these cases, is zero loss always possible? If not, why not? If sometimes, give examples of data distributions where zero loss is or is not possible.

Solution. (a) $H > D$: It is always possible. Basically we could find a large enough H such that the H hidden neurons forms a new feature space of higher dimension. For example, if we have data matrix $\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ where $D = 2, N = 2$, weight matrices $\mathbf{V} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$ and $\mathbf{W} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$ and RELU activation function $\max_x\{x, 0\}$, then the neural network would output $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ for input $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ for input $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ after training process, which is a zero loss case.

(b) $H = D$: This is sometimes possible, as is in the case of our XOR problem in the reading. But for some problem it is not.

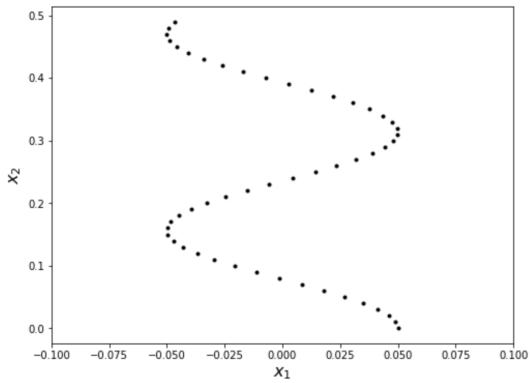
i. If we have data matrix $\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ where $D = 2, N = 2$, weight matrices $\mathbf{V} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ and $\mathbf{W} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ and RELU activation function $\max_x\{x, 0\}$, then the neural network would output $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ for input $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ for input $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ after training process, which is a zero loss case.

ii. If we have data such that is slightly more complicated, we may not reconstruct the input as expected. For example, if we create a dataset of random floating-point numbers, normalizes them to be between 0 and 1, and then adds Gaussian noise to create the noisy inputs. If we train the model above with $H = 10, D = 10$, we will never converge to 0 loss. Code and plot are in the appendix where we can see that the loss won't converge to 0 but still fluctuates.

(c) $H < D$: This is always not possible. Just like in PCA, we lose some information about the original data. Even if we introduce some non-linearity in neural network, the loss of information cannot be recovered.

□

(D) How does our choice of a affect our neural network's performance? Consider two activation functions: (i) $a(x) = \frac{1}{1+\epsilon^{-x}}$ (the sigmoid function), and (ii) $a(x) = x$ (the identity function). For the 2D input data described by the plot below, what might the hidden representation $\mathbf{h}$ look like for each of these activation functions? Why?



Solution. (a) For the sigmoid function, each dimension of the hidden layer $h_i, \forall i \in \{1, 2, \cdots, H\}$ will be clustered within the range (0,1), which could cause the variation in these points to be very small and we know that the less the variation in the data the less information it contains. Thus, using sigmoid will cause the loss of information of the input data.

(b) For the identity function, the hidden representation h would be a linear transformation of the input data. Since linear transformation can be interpreted as rotation and stretching(SVD), we expect the hidden representation to be a rotated and scaled version(no shifting since there is no bias) of the plot above.

□

4 Building a Latent Variable Model

In this question, you will build a latent variable model by making certain assumptions about your data. Now, you have data about N patients that were treated with a new drug to control their blood cholesterol (BC) level. For each patient $1 \leq i \leq N$ in the dataset, you observe a single scalar change in BC and no other data; let us denote this value as x_i. A biologist collaborator tells you that there potentially exists K patient characteristics that affect how BC changes due to drug intake. Since these factors are unmeasured, you build a model with latent variables $z_{i,k}$ that indicate the presence of characteristic k for patient i.

Denote the characteristic vector for patient i as $\mathbf{z}_i = [z_{i,1}, \cdots, z_{i,K}]$ where $z_{i,k} \in \{0, 1\}$ (i.e. assume that the latent variables are binary). Furthermore, assume that $(x_i, \mathbf{z}_i) \perp\!\!\!\perp (x_j, \mathbf{z}_j)$ for $i \neq j$.

(A) Choose a prior distribution for the characteristic vectors $\mathbf{z}_i \sim p(\mathbf{z})$ and justify your choice, i.e. describe the assumptions that you made.

Solution. We could model each $z_{i,j} \forall j \in \{1, 2, \cdots, K\}$ as i.i.d binary bernoulli random variables such that $\mathbf{z}_i \sim \prod_{j=1}^{K} p_j^{z_{i,j}}(1-p_j)^{1-z_{i,j}}$ Here we assume that:

(a) For each patient i, the presence of one character is independent of the presence of another.

(b) For each patient i, the probability that each characteristics is present is the same, which means $p_j = p, \forall j = 1, 2, \cdots, K$.

□

(B) The biologist tells you that given any patient's characteristics $\mathbf{z}_i$, the observed BC can be modeled as:

$$v_{i,k} \sim \mathcal{N}(\mu_k, 1), \qquad \epsilon_i \sim \mathcal{N}(0, 1), \quad x_i = \sum_{k=1}^{K} z_{i,k} v_{i,k} + \epsilon_i$$

where $\{\mu_k\}_{1 \leq k \leq K}$ are known quantities. Write down the the likelihood distribution $p(x_i \mid \mathbf{z}_i)$ explicitly.

Solution. Assume that $v_{i,k}, \epsilon_i$ are independent, then the sum of these random variables are still gaussian, where $\mu_i = \sum_{k=1}^{K} z_{i,k}\mu_k$ and $\sigma_i^2 = \sum_{k=1}^{K} z_{i,k}^2 + 1$. Without the assumption, we cannot make this argument. Then the likelihood distribution is defined as

$$p(x_i|\mathbf{z}_i) \sim \mathcal{N}(\sum_{k=1}^{K} z_{i,k}\mu_k, \sum_{k=1}^{K} z_{i,k}^2 + 1)$$

□

(C) Under your choice of prior in part (A) and the likelihood specified by the biologist in part (B), write down the implied marginal distribution $p(x_i)$ in terms of known quantities like $\{\mu_k\}_{1 \leq k \leq K}$. How many modes can this distribution have?

Solution. By the bayes rule we have

$$p(x_i) = \sum_{\mathbf{z}_i \in \mathcal{Z}_i} p(x_i|\mathbf{z}_i)p(\mathbf{z}_i)$$

since we know from part (A) that $\mathbf{z}_i$ is an indicator vector where each entry is binary, the cardinality of $\mathcal{Z}_i$ would be 2^K. So for each patient i, we can have 2^K different marginal distribution for x_i, which is just the modes of the distribution of x_i. □

(D) How would you infer $\mathbf{z}_i$ for any patient i? Write down one way to do it in terms of observed data $\{x_i\}_{1\leq i\leq N}$ and known quantities $\{\mu_k\}_{1\leq k\leq K}$.

Solution. We could utilize bayes rule and MAP to infer the hidden variables, which is based on:

$$p\left(\mathbf{z}_i \mid x_i\right) \propto p\left(x_i \mid \mathbf{z}_i\right) p\left(\mathbf{z}_i\right)$$

In essence, we want to compute the $\operatorname{argmax}_{\mathbf{z}_i} p(\mathbf{z}_i|x_i)$ where $\mathbf{z}_i \in \mathcal{Z}_i$ and $|\mathcal{Z}_i| = 2^K$. □

(E) Now consider a different likelihood model than in (B). Instead of giving you exact values, suppose that the biologist now tells you a range for each μ_k in $\{\mu_k\}_{1\leq k\leq K}$. What prior would you place on these means and why?

Solution. I would choose gaussian prior for the following reasons:

(a) If someone tells you a range of values they believe to be acceptable, then we could use gaussian prior to describe the characteristics of the range. The value in the range that is believed to be the most likely is parametrized by μ_k. To be specific, one can model the uncertainty in the parameter by specifying the variance of μ_k and the most likely value of the parameter by specifying the mean of μ_k.

(b) Gaussian prior has lots of conjucacy relationship with lots of otherwise distributions(including itself).

(c) Computing the log-likelihood related to gaussian distribution is numerically stable and computationally easy.

□

(F) Now consider yet another likelihood model, where we let each possible value of $\mathbf{z}_i$ parameterize the likelihood with a separate mean $\mu_{\mathbf{z}_i} = \mathbb{E}[x_i|\mathbf{z}_i]$? What are the trade-offs of this model compared to our original one in (B)? For example, what happens when K is large?

Solution. This model doesn't assume the likelihood to be an additive gaussian model, and doesn't assume the same gaussian distribution for all possible configuration of $\mathbf{z}_i$. This brings in more flexibility in the model but at the cost of increased model complexity. The reason is that, former model only has one mean μ_k for all possible 2^K configuration of $\mathbf{z}_i$ but the new model has 2^K such μ_k, which grows exponentially as $K \to \infty$. Thus the trade off of this model is the increased model flexibility at the cost of high overfitting probability. □

5 Bias-Variance Tradeoff and Regularization

In this question, you will study the bias and variance of the linear regression estimator with and without ℓ_2 regularization. The input points $\mathbf{x}_i$ are P dimensional and are sampled i.i.d. from $\mathcal{N}(0, I_P)$. The targets are given by $y_i = \mathbf{x}_i^T \boldsymbol{\theta}^* + \epsilon_i$ where the noise ϵ_i are sampled from $\mathcal{N}(0, \sigma^2)$. We have access to $N = P + 1$ data points.

(A) We are given an estimator $\boldsymbol{\theta}$. Express the *risk* $R(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}\sim\mathcal{N}(0,I_P),\ y|\mathbf{x}\sim\mathcal{N}(\mathbf{x}^T\boldsymbol{\theta}^*,\sigma^2)}[(\mathbf{x}^T\boldsymbol{\theta} - y)^2]$ in terms of bias and variance of the estimator and the noise terms. Indicate what each term corresponds to.

Solution. Denote. $\vec{\theta}^\top \vec{x} = h(\vec{x}; D)$, and $E_{\vec{x}\sim N(0,I_p),y|\vec{x}\sim N\left(\vec{x}^\top\vec{\theta}^*,\sigma^2\right)}[\cdot] = E[\cdot]$ where D is the training set(viewed as a random variable).

$$\begin{aligned}
E\left[(h(\vec{x}, D) - y)^2\right] &= E\left[h(\vec{x}; D)^2\right] - 2E[h(\vec{x} : D) \cdot y] + E\left[y^2\right] \\
&= \mathrm{Var}[h(\vec{x}; D)] + E(h(\vec{x}; D)]^2 - 2E(h(\vec{x}; D) \cdot y) + \mathrm{Var}[y] + E[y]^2 \\
&= \mathrm{Var}(h(\vec{x}; D)] + E[h(\vec{x}; D)]^2 - 2E[h(\vec{x}; D)] \cdot E[y] + E[y]^2 + \mathrm{Var}(y] \\
&= \underbrace{\mathrm{Var}(h(\vec{x}; D)]}_{\text{Variance}} + \underbrace{(E\left(\vec{x}; D)\right] - E(y))^2}_{\text{Bias}} + \underbrace{\mathrm{Var}(\varepsilon)}_{\text{Irreducible error.}}
\end{aligned}$$

Irreducible error Here, since $\vec{x}$ and y is our test point, which are independent of the training dataset D $\therefore g(D)$ and y are independent (functions of independent R, V, s are independat)

$$\therefore E[h(\vec{x}; D) \cdot y] = E[h(\vec{x}; D)]E[y]$$

Also $\mathrm{Var}[y] = \mathrm{Var}(\varepsilon)$ since $y \sim N\left(\vec{\theta}^\top \vec{x}, \sigma^2\right)$. Now we fit in with the value under our model assumptions:

$$\begin{aligned}
\mathrm{Var}(h(\vec{x}; D)) &= \mathrm{Var}\left(\vec{x}^\top \vec{\theta}\right) \\
\left(E(h(\vec{x}; D) - E(y)\right)^2 &= \left(E_x\left[\vec{\theta}^\top \vec{x} - y\right]\right)^2 \\
&= \left(E_X\left[E_{Y|X}\left[\theta^\top \vec{x} - y\right]\right]\right)^2 \\
&= \left(E_X\left[\vec{x}^\top \vec{\theta} - \vec{x}^\top \vec{\theta}^*\right]\right)^2 \\
\mathrm{Var}(\varepsilon) &= \sigma^2
\end{aligned}$$

□

(B) Use a ridge parameter $\lambda \geq 0$, i.e. the coefficient of ℓ_2 regularization. Express the estimator $\boldsymbol{\theta}$ learned with ridge regression in terms of the data matrix $X = [\mathbf{x}_1, ..., \mathbf{x}_N] \in \mathbb{R}^{P\times N}$, the noise vector $E = [\epsilon_1, ..., \epsilon_N] \in \mathbb{R}^N$, the true parameter $\boldsymbol{\theta}^*$, and the ridge λ.

Solution. We immediately know from l_2 regularization least squared solution that:

$$\begin{aligned}
\hat{\theta} &= \left(XX^\top + \lambda I_p\right)^{-1} XY \\
&= \left(XX^\top + \lambda I_p\right)^{-1} X\left(X^\top \theta^* + E\right)
\end{aligned}$$

□

(C) Give the formulae of the bias $\mathbb{E}_{X,E}[\boldsymbol{\theta}] - \boldsymbol{\theta}^*$ and variance $\mathbb{E}_{X,E}[\boldsymbol{\theta}^2] - \mathbb{E}_{X,E}[\boldsymbol{\theta}]^2$ of the ridge regression estimator $\boldsymbol{\theta}$. The formulae should be expressed in terms of the ridge λ and as an expectation over X.

Solution.

$$\begin{aligned}
\text{Bias} &= E_{X,E}\left[\left(XX^\top+\lambda I_p\right)^{-1}X\left(X^\top\theta^*+E\right)-\theta^*\right]\\
&= E_{X,E}\left[\left(XX^\top+\lambda I_p\right)^{-1}XX^\top\theta^*\right]+E_{X,E}\left(\left(XX^\top+\lambda Z_p\right)^{-1}XE\right]-\theta^*\\
&= E_X\left[\left(XX^\top+\lambda I_p\right)^{-1}XX^\top\right]\theta^*-\theta^*\\
\text{Covariance Matrix} &= E_{X,E}\left[\left(\hat\theta-\theta^*\right)\left(\hat\theta-\theta^*\right)^\top\right]\\
&= E_{X,E}\left(\left(XX^\top+\lambda I_p\right)^{-1}\left(XY'-XX^\top\theta^*\right)\left(XY-XX^\top\theta^*\right)^\top\left(XX^\top+\lambda I_p\right)^{-\top}\right]\\
&= E_{X,E}\left[\left(XX^\top+\lambda I_p\right)^{-1}XE(XE)^\top\left(XX^\top+\lambda I_p\right)^{-1}\right]\\
&= E_X\left[\left(XX^\top+\lambda I_p\right)^{-1}XX^\top\left(XX^\top+\lambda I_p\right)^{-1}\right]E_E\left[EE^\top\right]\\
&= E_X\left[\left(XX^\top+\lambda I_p\right)^{-1}XX^\top\left(XX^\top+\lambda I_p\right)^{-1}\right]\\
\textit{Variance} &= \text{Tr(Covariance Matrix)}\\
&= E_X\left[\text{Tr}\left[\left(XX^\top+\lambda I_p\right)^{-1}XX^\top\left(XX^\top+\lambda I_p\right)^{-1}\right]\right.
\end{aligned}$$

□

(D) Set $\lambda = 0$. In this ridgeless case, is the estimator unbiased? Express the variance in terms of the eigenvalues of the matrix (XX^T) and comment on its behavior when the number of features is large. *Hint: Marchenko-Pastur distribution describes the eigenvalues of XX^T when P is large. Commenting on the behavior of the minimum eigenvalue will suffice to explain the behavior of the variance.*

Solution. When $\lambda = 0$

$$\begin{aligned}
\text{Bias} &= E_x\left[\left(XX^\top+\lambda I_p\right)^{-1}XX^\top\right]\theta^*-\theta^*\\
&= E_x\left[\left(XX^\top\right)^{-1}XX^\top\right]\theta^*-\theta^*\\
&= \theta^*-\theta^*\\
&= 0\\
\text{CoV} &= E_X\left[\left(XX^\top+\lambda Z_P\right)^{-1}XX^\top\left(XX^\top+\lambda Z_P\right)^{-1}\right]E_E\left[EE^\top\right]\\
&= E_X\left[\left(XX^\top\right)^{-1}XX^\top\left(XX^\top\right)^{-1}\right]E_E\left[EE^\top\right]\\
&= E_X\left[\left(XX^\top\right)^{-1}\right]E_E\left[EE^\top\right]\\
&= E_X\left[\left(XX^\top\right)^{-1}\right]
\end{aligned}$$

$$\begin{aligned}
\text{Variance} = \text{Tr[cov]}\\
&= E_X\left[\text{Tr}\left(XX^\top\right)^{-1}\right]\\
&= E_X\left[\sum_{i=1}^{p}\frac{1}{\sigma_i^2}\right]
\end{aligned}$$

where σ_i is the sigular value of X and σ_i^2 is the eigenvalues of $XX^\top$

By Marchanko- Pasture law, we have

$$\sigma_i^2 \in \left[1 - \sqrt{\frac{P}{P+1}}, 1 + \sqrt{\frac{P}{P+1}}\right]$$

$\therefore \sigma_{\min}^2$ will be close to 0 since $\sqrt{\frac{P}{P+1}} \to 1$ as $P \to \infty$

$\therefore \frac{1}{\sigma_{\min}^2} \longrightarrow \infty$, causing the variance to explode. □

(E) To mitigate the large variance, we want to use a non-zero ridge. In this ridge case, is the estimator unbiased? Use the formulae of the variance to explain how the ridge parameter mitigates the large variance.

Solution. When $\lambda \neq 0$

$$\begin{aligned}
\text{Variance} &= \text{Tr}(\text{ Covariance Matrix }) \\
&= E_X\left[\text{Tr}\left[\left(XX^\top + \lambda I_P\right)^{-1} XX^\top \left(XX^\top + \lambda I_P\right)^{-1}\right]\right. \\
&= E_X\left[\sum_{i=1}^{P} \frac{\sigma_i^2}{\left(\sigma_i^2 + \lambda\right)^2}\right]
\end{aligned}$$

Where σ_i^2 is the eigenvalues of $XX^\top$. As $P \to \infty$, by the theorem above, the smallest eigenvalue $\sigma_{\min}^2$ will be incremented by λ, preventing high variance in this direction

$$\begin{aligned}
\text{Bias} &= E_{X,E}\left[\left(XX^\top + \lambda I_p\right)^{-1} X\left(X^\top\theta^* + E\right) - \theta^*\right] \\
&= E_{X,E}\left[\left(XX^\top + \lambda I_P\right)^{-1} XX^\top\theta^*\right] + E_{X,E}\left(\left(XX^\top + \lambda I_P\right)^{-1} XE\right] - \theta^* \\
&= E_X\left[\left(XX^\top + \lambda I_P\right)^{-1} XX^\top\right]\theta^* - \theta^* \neq \theta^*
\end{aligned}$$

□

6 Neural Networks and Overparameterization

In class, we learned that having more parameters than needed to perfectly learn a training dataset $(\mathbf{x}_i, y_i)$ for $i = 1, ..., N$ facilitates the optimization problem. Consider the dataset $(\mathbf{x}_1 = (-1, 1), y_1 = 0), (\mathbf{x}_2 = (1, -1), y_2 = 0), (\mathbf{x}_3 = (1, 1), y_3 = 1), (\mathbf{x}_4 = (-1, -1), y_4 = 1)$.

(A) Construct parameters of a neural network with two neurons and ReLU link function $\mathbf{x} \to \max(\boldsymbol{\theta}_1^T \mathbf{x} + b_1, 0) + \max(\boldsymbol{\theta}_2^T \mathbf{x} + b_2, 0)$ that achieves perfect accuracy on the training data. You can either write out the parameters $(\boldsymbol{\theta}_i, b_i)$ explicitly, or draw the hyperplanes $\boldsymbol{\theta}_i^T \mathbf{x} + b_i = 0$ corresponding to each neuron in the input space. Explain how your neural network achieves perfect accuracy.

Solution. We could choose $\boldsymbol{\theta}_1$ to be $\begin{bmatrix} 1 & 1 \end{bmatrix}^\top$ and $\boldsymbol{\theta}_2$ to be $\begin{bmatrix} -1 & -1 \end{bmatrix}^\top$ while $b_1 = -1, b_2 = 0$. Then the output of $\mathbf{x}_1$ and $\mathbf{x}_2$ would be 0 while the output of $\mathbf{x}_3$ and $\mathbf{x}_4$ would be 1, achieving perfect accuracy. □

Use parameters you constructed in (A) to generate new parameters in the overparameterized neural network with $H \geq 2$ neurons by splitting neurons

$$\max(\boldsymbol{\theta}_i^T \mathbf{x} + b_i, 0) \to \max(\alpha \boldsymbol{\theta}_i^T \mathbf{x} + \alpha b_i, 0) + \max((1 - \alpha)\boldsymbol{\theta}_i^T \mathbf{x} + (1 - \alpha) b_i, 0)$$

for some $\alpha \in [0, 1]$. The new parameter vector after neuron splitting generates the same neural network function. Substitute parameters of each one of H neurons by using either one of the two neurons' parameters. Consider fixed α's after splitting neurons.

(B) How many such parameter vectors are there? The collection of neural network parameters is a $3H$ dimensional vector. How does the number of the parameter vectors scale compare to the number of the parameters when H is large?

Solution. (a) There will be 2^H such parameter vectors there since for each of H neurons, it can be set to either $\max\left(\alpha \boldsymbol{\theta}_i^T \mathbf{x} + \alpha b_i, 0\right)$ or $\max\left((1 - \alpha)\boldsymbol{\theta}_i^T \mathbf{x} + (1 - \alpha) b_i, 0\right)$, two choices for each hidden neural. Meanwhile, each parameter vector is a $3H$ dimension vector, 2 for weights and 1 for bias.

(b) When H increases, the number of parameter vectors grows exponentially while the number of parameters grows linearly with the H.

□

(C) Using your answer in part (B), compare the cases of no overparameterization ($H = 2$) vs infinite overparameterization ($H \to \infty$) and comment on how overparameterization facilitates the optimization problem.

Solution. (a) Without overparametrization, the neural network's representation ability will be greatly limited. At the cost of underfitting, the neural network could produce more generalizable output. However, such generalizability can be caused by the difficulties in the training step. With less parameters, it will be harder for the gradient-based optimizer to find the optimal descent direction efficiently and could stop at some local minima.

(b) With overparametrization($H \to \infty$), the first-order gradient-based method will become more easily to converge to the optimal point since there are more descent directions to choose from. Moreover, the model's representation ability will be greatly improves, it can approximate wider range of functions. However, the overparametrization could cause the notorious overfitting problem.

(c) Recent papers suggest that there will be a so-called double descent phenomenon as we gradually increase the model complexity by increasing the number of parameters. In other words, the test error of the trained neural network will first decrease then increase to the point where the parameters are enough to fit all the data points well. After that point, if we keep increasing the model complexity, the test error will decrease again and never ascent again in the future.

□

A Codes

```
import torch
import torch.nn as nn

# Define the dimensions of the input and hidden layer
D = 10
H = 10

# Define a simple autoencoder class
class SimpleAutoencoder(nn.Module):
        def __init__(self, input_dim, hidden_dim):
                super(SimpleAutoencoder, self).__init__()
                self.encoder = nn.Linear(input_dim, hidden_dim)
                self.decoder = nn.Linear(hidden_dim, input_dim)
                self.activation = nn.ReLU()

        def forward(self, x):
                h = self.activation(self.encoder(x))
                x_reconstructed = self.decoder(h)
                return x_reconstructed

# Initialize the network
autoencoder = SimpleAutoencoder(D, H)
```

```
SimpleAutoencoder(
  (encoder): Linear(in_features=10, out_features=10, bias=True)
  (decoder): Linear(in_features=10, out_features=10, bias=True)
  (activation): ReLU()
)
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import numpy as np

class NoisyDataset(Dataset):
        def __init__(self, num_samples=1000, input_dim=10, noise_factor=0.5):
                self.num_samples = num_samples
                self.input_dim = input_dim
                self.noise_factor = noise_factor

                # Generate synthetic data
                self.data = np.random.rand(num_samples, input_dim).astype(np.float32)

                # Normalize the data to be between 0 and 1
                self.data = (self.data - np.min(self.data)) / (np.max(self.data) - np.min(self.data))

        def __len__(self):
                return self.num_samples

        def __getitem__(self, index):
                clean_data = self.data[index]
                noisy_data = clean_data + self.noise_factor * np.random.randn(self.input_dim).astype(np.float32)
                noisy_data = np.clip(noisy_data, 0, 1)    # Clip values to be between 0 and 1
                return clean_data, clean_data


dataset = NoisyDataset(num_samples=1000, input_dim=10, noise_factor=0.5)


# Hyperparameters
num_epochs = 500
batch_size = 64
learning_rate = 1e-3

# Dataloader defined here
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

```

```
# Loss function
criterion = nn.MSELoss()

# Optimizer
optimizer = optim.Adam(autoencoder.parameters(), lr=learning_rate)

loss_epoch = []
# Training loop
for epoch in range(num_epochs):

        for data in dataloader:
                inputs,_ = data    # We don't need the labels for autoencoder training
                # Forward pass
                outputs = autoencoder(inputs)
```

```
            loss = criterion(outputs, inputs)   # Calculate the loss between the output and the original inputs

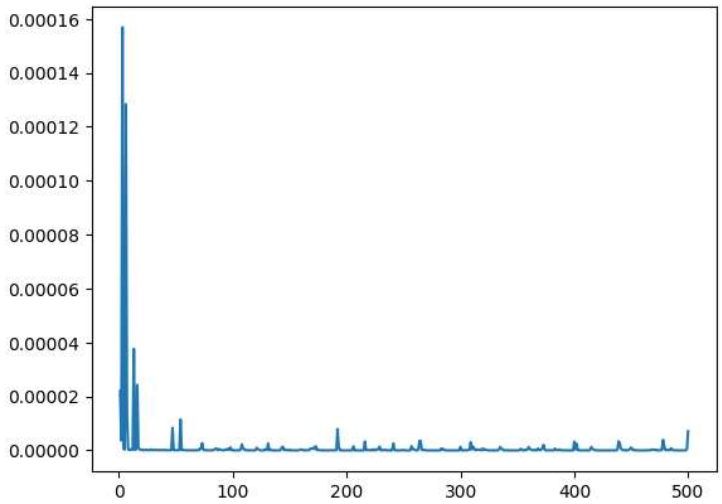
            # Backward and optimize
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        loss_epoch.append(loss.item())

loss_epoch = np.array(loss_epoch)
```

```
import matplotlib.pyplot as plt
plt.plot(range(1, len(loss_epoch) + 1), loss_epoch)
```

```
[<matplotlib.lines.Line2D at 0x7cc2b610f2e0>]
```




```
loss_epoch
```

```
array([2.86776483e-01, 2.07917705e-01, 1.42580301e-01, 1.12540700e-01,
       9.34251323e-02, 8.52063447e-02, 7.15045333e-02, 7.66555518e-02,
       7.47427791e-02, 6.65282533e-02, 6.27608523e-02, 6.51037768e-02,
       5.96492402e-02, 6.00489266e-02, 6.25638366e-02, 5.53774126e-02,
       5.20703606e-02, 5.18064462e-02, 4.56837639e-02, 4.24508899e-02,
       4.10954468e-02, 4.31896262e-02, 3.16594876e-02, 3.22170369e-02,
       3.70135829e-02, 3.16844955e-02, 2.69386284e-02, 2.90569551e-02,
       2.76037119e-02, 2.92315315e-02, 2.45926995e-02, 2.63987035e-02,
       2.07602568e-02, 2.17636768e-02, 2.51242667e-02, 2.19227262e-02,
       2.18258146e-02, 2.11281199e-02, 1.96457785e-02, 1.94821265e-02,
       1.72192659e-02, 1.66226681e-02, 1.99205000e-02, 1.69491600e-02,
       1.79326031e-02, 1.37800630e-02, 1.54458620e-02, 1.58474818e-02,
       1.28772296e-02, 9.60879494e-03, 1.04945675e-02, 1.24802962e-02,
       1.11866686e-02, 1.44624747e-02, 1.04988096e-02, 8.75800103e-03,
       9.01759043e-03, 9.95622203e-03, 7.27449777e-03, 9.69430432e-03,
       8.69701058e-03, 8.88047554e-03, 7.80762499e-03, 1.09641468e-02,
       7.10470136e-03, 7.98077695e-03, 7.74636446e-03, 7.35446671e-03,
       8.02850351e-03, 6.97876979e-03, 1.02161476e-02, 8.34182836e-03,
       7.82401860e-03, 6.03429088e-03, 8.08303617e-03, 6.55380078e-03,
       6.91429665e-03, 8.16041697e-03, 6.14794623e-03, 6.75297715e-03,
       5.00594033e-03, 7.47516751e-03, 5.05219260e-03, 5.39846579e-03,
       5.66560216e-03, 5.13133267e-03, 4.71440097e-03, 6.60183793e-03,
       6.33152667e-03, 4.08747001e-03, 4.16569458e-03, 4.10021702e-03,
       3.50055238e-03, 3.17945355e-03, 2.74306000e-03, 1.64315326e-03,
       2.74024135e-03, 2.31042528e-03, 2.37221224e-03, 1.77957234e-03,
       1.14653772e-03, 1.62392622e-03, 1.16167811e-03, 8.22812086e-04,
       9.72139591e-04, 6.36040291e-04, 6.52780174e-04, 3.91058740e-04,
       4.09510569e-04, 5.18103596e-04, 4.82374104e-04, 2.22399918e-04,
       2.09506616e-04, 1.49233412e-04, 1.81422787e-04, 1.46851584e-04,
       1.03174934e-04, 7.79778638e-05, 1.49457614e-04, 9.15085184e-05,
       5.87401337e-05, 4.83364602e-05, 1.12386922e-04, 3.36025078e-05,
       5.80580236e-05, 4.93122934e-05, 2.87793646e-05, 1.59581850e-05,
       2.11546958e-05, 7.94974039e-05, 3.27246817e-05, 3.32923300e-05,
       1.67143589e-04, 1.19848792e-05, 1.61031676e-05, 1.10011060e-05,
       2.19571630e-05, 3.75044474e-05, 7.71105188e-05, 1.96134788e-05,
       8.39746499e-05, 1.06752923e-05, 6.79444520e-06, 6.98006697e-05,
       4.37755807e-06, 6.71819435e-05, 4.95944187e-06, 1.32983687e-05,
       1.47892079e-05, 4.01131820e-06, 3.55105681e-06, 3.12229736e-06,
       3.51387371e-06, 6.61374215e-05, 2.16186818e-05, 2.64920004e-06,
       2.22427207e-05, 2.30669229e-06, 1.60597738e-05, 1.43286898e-05,
       2.24976102e-05, 4.82434416e-06, 1.25479746e-05, 1.33272306e-06,
       1.30932835e-06, 6.11022624e-05, 1.01097030e-05, 1.83341890e-05,
       1.61537207e-06, 1.86798134e-05, 1.73587603e-06, 1.61473156e-06,
       8.88614181e-07, 8.43058660e-06, 3.07669302e-06, 3.01814980e-05,
       2.91672459e-05, 2.42315691e-05, 5.94270750e-05, 1.43160287e-06,
       1.07576000e-06, 4.37015103e-07, 4.51728113e-07, 4.11408109e-07,
       1.04122319e-06, 8.99529482e-07, 2.56942343e-07, 2.10222424e-05,
```

```
3.54018312e-07,  1.34251222e-05,  1.65966012e-05,  1.39877593e-05,
1.56121441e-05,  5.18648358e-06,  2.30065348e-06,  6.91284367e-06,
1.64218747e-07,  1.03550847e-05,  6.64972276e-06,  1.27416642e-05,
1.79737126e-05,  1.13839697e-07,  9.50923095e-06,  1.61707817e-07,
1.08001746e-06,  8.45113561e-08,  1.30375980e-07,  6.22678371e-08,
7.06777428e-08,  9.80039729e-08,  1.12882083e-06,  1.48880383e-06,
2.46119384e-06,  5.75302010e-06,  3.41942223e-06,  5.88971218e-08,
6.31906190e-08,  5.57336790e-08,  6.28780512e-08,  2.55429109e-06,
1.11561421e-05,  4.23437143e-08,  3.78611526e-06,  5.50478205e-08,
4.15056002e-06,  3.75802358e-08,  3.35876575e-06,  8.46902276e-06,
3.34581091e-06,  3.87173849e-08,  3.40659341e-08,  2.71512477e-06,
```

1 开始借助 AI 编写或生成代码。